

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	T. Bhanu Pavan Varma	Reg. No.:	192425380
Section / Batch:	B. Tech AI&ML (2024-2028)	Date:	05/08/2026

Code of Conduct

I T. Bhanu Pavan Varma(192425380) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Part of Speech Tagging	Morphological decomposition of analyzing, analysis, and analytical; identification of roots, affixes, inflectional and derivational forms, and normalization for indexing.	Q1
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q2
Counting Words, English Word Classes, Part of Speech Tagging	Morphology-based normalization of govern, government, and governance for topic modeling, clustering, and lexical standardization.	Q3
Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Morphological analysis of activate, activation, and reactivation; identification of derivational layers, word-class changes, and semantic shifts.	Q4
Rule-Based Part of Speech Tagging, English Word Classes, Part of Speech Tagging	Inflectional normalization of create, creates, and creating; extraction of root forms and classification of tense/aspect transformations.	Q5

Annexure A – SCENARIO BASED

1. A large-scale search engine indexing system processes user queries containing word variants such as “analyzing”, “analysis”, and “analytical”. The system must improve retrieval accuracy by grouping semantically related terms under a unified representation. Design a morphological processing approach to decompose each input word into root and affixes, distinguish between inflectional and derivational forms, and normalize them for indexing. Apply the process to the given inputs and determine the root form along with the morphological structure of each word.

- Develop a Python program to implement a rule-based morphological processing system for the input words "analyzing", "analysis", and "analytical". The program should identify the root word and separate the corresponding prefixes or suffixes using suitable morphological rules, classify each word as an inflectional or derivational form, normalize all related variants to a common representation for indexing, and generate a structured report showing the original word, extracted root, identified affix(es), transformation type, and normalized output to support efficient search engine retrieval.

Experiment 1: Rule-Based Morphological Processing

```
words = ["analyzing", "analysis", "analytical"]
```

```
for word in words:
```

```
    root = ""
```

```
    affix = ""
```

```
    morph_type = ""
```

```
    if word.endswith("ing"):
```

```
        root = "analyze"
```

```
        affix = "-ing"
```

```
        morph_type = "Inflectional"
```

```
    elif word.endswith("sis"):
```

```
        root = "analyze"
```

```
        affix = "-sis"
```

```
        morph_type = "Derivational"
```

```
    elif word.endswith("ical"):
```

```
        root = "analyze"
```

```
        affix = "-ical"
```

```
        morph_type = "Derivational"
```

```
print("Original Word :", word)
```

```
print("Root      :", root)
```

```
print("Affix     :", affix)
```

```
print("Type      :", morph_type)
```

```
print("Normalized :", "analyze")
```

```
print("-" * 40)
```

```
● (base) bhanu@Bhanus-MacBook-Air C02-AT2 % python Q1.py
```

```
Original Word : analyzing
```

```
Root          : analyze
```

```
Affix         : -ing
```

```
Type          : Inflectional
```

```
Normalized    : analyze
```

```
-----
```

```
Original Word : analysis
```

```
Root          : analyze
```

```
Affix         : -sis
```

```
Type          : Derivational
```

```
Normalized    : analyze
```

```
-----
```

```
Original Word : analytical
```

```
Root          : analyze
```

```
Affix         : -ical
```

```
Type          : Derivational
```

```
Normalized    : analyze
```

```
-----
```

```
■ (base) bhanu@Bhanus-MacBook-Air C02-AT2 %
```

2. An AI-based sentiment analysis platform receives product reviews containing words like “disagree”, “agreement”, and “agreeable”. The system must interpret sentiment consistently despite variations in word formation. Develop a morphological parsing strategy to identify prefixes, suffixes, and base forms, while capturing how derivational changes influence meaning. Process the given inputs to extract roots and classify each transformation, ensuring accurate semantic interpretation across all word forms.

- Develop a Python program to create a morphological parser for the input words "disagree", "agreement", and "agreeable". The program should detect and separate prefixes, suffixes, and the base word using rule-based morphological analysis, examine how derivational modifications alter the meaning of the root word, classify each transformation according to its morphological type, and produce a comprehensive output containing the original word, identified prefix, root word, suffix, transformation category, semantic interpretation, and normalized base form for use in sentiment analysis applications.

Experiment 2: Morphological Parsing

```
words = ["disagree", "agreement", "agreeable"]
```

```
for word in words:
```

```
    prefix = ""
```

```
    suffix = ""
```

```
    root = "agree"
```

```
    morph_type = ""
```

```
    meaning = ""
```

```
    if word.startswith("dis"):
```

```
        prefix = "dis-"
```

```
        morph_type = "Derivational"
```

```
        meaning = "Negative meaning"
```

```
    elif word.endswith("ment"):
```

```
        suffix = "-ment"
```

```
        morph_type = "Derivational"
```

```
        meaning = "State or result"
```

```
    elif word.endswith("able"):
```

```
        suffix = "-able"
```

```
        morph_type = "Derivational"
```

```
        meaning = "Capable of"
```

```
    print("Original Word :", word)
```

```
    print("Prefix      :", prefix)
```

```
    print("Root       :", root)
```

```
    print("Suffix     :", suffix)
```

```
    print("Type       :", morph_type)
```

```
    print("Meaning    :", meaning)
```

```
    print("Normalized :", root)
```

```
    print("-" * 40)
```

```
● (base) bhanu@Bhanus-MacBook-Air C02-AT2 % python Q2.py
```

```
Original Word : disagree
Prefix       : dis-
Root        : agree
Suffix      : -
Type       : Derivational
Meaning    : Negative meaning
Normalized : agree
```

```
-----
Original Word : agreement
Prefix       : -
Root        : agree
Suffix      : -ment
Type       : Derivational
Meaning    : State or result
Normalized : agree
```

```
-----
Original Word : agreeable
Prefix       : -
Root        : agree
Suffix      : -able
Type       : Derivational
Meaning    : Capable of
Normalized : agree
```

```
❏ (base) bhanu@Bhanus-MacBook-Air C02-AT2 %
```

3. A text analytics pipeline in a news aggregation system handles documents containing terms such as “govern”, “government”, and “governance”. The system must standardize these variations for topic modeling and clustering. Construct a morphology-based normalization mechanism that decomposes each word into its root and affixes, identifies derivational layers, and maps them to a common base form. Apply the approach to the input words and derive the normalized representation along with their morphological breakdown.

- Develop a Python program to implement a morphology-based normalization module for the input words "govern", "government", and "governance". The program should perform morphological decomposition to identify the root word and successive derivational suffixes, determine the derivational level associated with each word, normalize all variants to a common lexical representation, and generate a structured output displaying the original word, root form, detected affix(es), derivational hierarchy, normalized representation, and the final output used for topic modeling and document clustering.

Experiment 3: Morphological Normalization

```
words = ["govern", "government", "governance"]
```

```
for word in words:
```

```
    root = "govern"
```

```
    suffix = "-"
```

```
    level = "Base"
```

```
    if word == "government":
```

```
        suffix = "-ment"
```

```
        level = "Level 1"
```

```
    elif word == "governance":
```

```
        suffix = "-ance"
```

```
        level = "Level 1"
```

```
    print("Original Word :", word)
```

```
    print("Root      :", root)
```

```
    print("Suffix     :", suffix)
```

```
    print("Hierarchy   :", level)
```

```
    print("Normalized  :", root)
```

```
    print("-" * 40)
```

```
• (base) bhanu@Bhanus-MacBook-Air C02-AT2 % python Q3.py
```

```
Original Word : govern
```

```
Root          : govern
```

```
Suffix        : -
```

```
Hierarchy     : Base
```

```
Normalized    : govern
```

```
Original Word : government
```

```
Root          : govern
```

```
Suffix        : -ment
```

```
Hierarchy     : Level 1
```

```
Normalized    : govern
```

```
Original Word : governance
```

```
Root          : govern
```

```
Suffix        : -ance
```

```
Hierarchy     : Level 1
```

```
Normalized    : govern
```

```
❖ (base) bhanu@Bhanus-MacBook-Air C02-AT2 %
```

4. A document classification system processes input words such as “activate”, “activation”, and “reactivation” to improve semantic grouping and indexing.

Construct a morphological parsing strategy to identify prefixes, suffixes, and root forms while distinguishing derivational layers.

Analyze how prefix addition and suffix transformation affect meaning and word class.

Apply the process to decompose each word and map them to a unified base representation.

Derive the morphological structure and normalized root for each input word.

- Develop a Python program to implement a morphological parsing and normalization system for the input words "activate", "activation", and "reactivation". The program should detect and separate prefixes, root words, and suffixes using rule-based morphological parsing, identify the sequence of derivational transformations that produce each word, analyze the effect of each morphological modification on the word class and semantic meaning, and generate a structured report showing the original word, prefix (if present), root word, suffix, derivational sequence, normalized base form, and final parsed representation for use in document classification and semantic indexing.

Experiment 4: Morphological Parsing

```
words = ["activate", "activation", "reactivation"]
```

```
for word in words:
```

```
    prefix = "-"
```

```
    suffix = "-"
```

```
    root = "activate"
```

```
    sequence = ""
```

```
    if word == "activate":
```

```
        sequence = "Base Form"
```

```
    elif word == "activation":
```

```
        suffix = "-ion"
```

```
        sequence = "activate + ion"
```

```
    elif word == "reactivation":
```

```
        prefix = "re-"
```

```
        suffix = "-ion"
```

```
        sequence = "re + activate + ion"
```

```
print("Original Word :", word)
```

```
print("Prefix      :", prefix)
```

```
print("Root       :", root)
```

```
print("Suffix     :", suffix)
```

```
print("Sequence   :", sequence)
```

```
print("Normalized  :", root)
```

```
print("-" * 40)
```

```
(base) bhanu@Bhanus-MacBook-Air C02-AT2 % python Q4.py
Original Word : activate
Prefix       : -
Root        : activate
Suffix      : -
Sequence    : Base Form
Normalized   : activate
-----
Original Word : activation
Prefix       : -
Root        : activate
Suffix      : -ion
Sequence    : activate + ion
Normalized   : activate
-----
Original Word : reactivation
Prefix      : re-
Root       : activate
Suffix    : -ion
Sequence  : re + activate + ion
Normalized : activate
-----
❖ (base) bhanu@Bhanus-MacBook-Air C02-AT2 %
```

5. A search optimization module processes input words such as “create”, “creates”, and “creating” to ensure consistent retrieval across grammatical variations. Design a morphology-based normalization approach focusing on inflectional transformations such as tense and aspect.

Identify suffix patterns and map all variations to a common base form without altering meaning.

Apply the process to extract root forms and classify each transformation.

Determine the morphological breakdown and normalized output for each word.

- Develop a Python program to implement an inflectional morphology-based normalization module for the input words "create", "creates", and "creating". The program should recognize grammatical suffixes corresponding to different inflectional forms, identify the underlying root word by applying appropriate normalization rules, classify each input according to its grammatical feature (e.g., base form, third-person singular, present participle), and produce a structured output containing the original word, identified suffix, grammatical category, extracted root, normalized base form, and the final normalized representation to support efficient search optimization and information retrieval.

Experiment 5: Inflectional Morphology Normalization

```
words = ["create", "creates", "creating"]
```

```
for word in words:
```

```
    suffix = "-"
```

```
    category = ""
```

```
    if word == "create":
```

```
        category = "Base Form"
```

```
    elif word.endswith("s"):
```

```
        suffix = "-s"
```

```
        category = "Third Person Singular"
```

```
    elif word.endswith("ing"):
```

```
        suffix = "-ing"
```

```
        category = "Present Participle"
```

```
    print("Original Word :", word)
```

```
    print("Suffix      :", suffix)
```

```
    print("Category   :", category)
```

```
    print("Root       :", "create")
```

```
    print("Normalized  :", "create")
```

```
    print("-" * 40)
```

```
● (base) bhanu@Bhanus-MacBook-Air C02-AT2 % python Q5.py
```

```
Original Word : create
```

```
Suffix       : -
```

```
Category     : Base Form
```

```
Root         : create
```

```
Normalized   : create
```

```
-----
```

```
Original Word : creates
```

```
Suffix       : -s
```

```
Category     : Third Person Singular
```

```
Root         : create
```

```
Normalized   : create
```

```
-----
```

```
Original Word : creating
```

```
Suffix       : -ing
```

```
Category     : Present Participle
```

```
Root         : create
```

```
Normalized   : create
```

```
-----
```

```
❖ (base) bhanu@Bhanus-MacBook-Air C02-AT2 %
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SCENARIO BASED ANALYSISQ1. MORPHOLOGY PROCESSING FOR SEARCH ENGINE :

A Search engine receives different forms of same words such as analyzing, analysis and analytical. If these words are stored separately, the search engine may miss relevant documents. Therefore, all related words must be normalized to a common base form.

INPUT WORDS :

- Analyzing
- analysis
- analytical

MORPHOLOGICAL ANALYSIS

WORD	ROOT	AFFIX	TYPE	NORMALIZED
analysis	analyze	sis	Derivational	analysis
analyzing	analyze	ing	Inflectional	analyze
analytical	analyze	tical	Derivational	analyze

OUTPUT :

All three words are normalized to analyze to improve search accuracy.

Q2) MORPHOLOGICAL PARSING FOR SENTIMENT ANALYSIS

A sentiment analysis system receives word like disagree, agreement and agreeable. The system must identify the base word and understand how prefixes and suffixes changes the meaning.

INPUT WORDS :

- Disagree
- Dis agreement
- agreeable

MORPHOLOGICAL ANALYSIS :

WORD	PREFIX	ROOT	SUFFIX	TYPE	MEANING
Disagree	dis	agree	-	Derivational	-ve meaning
agreement	-	agree	ment	Derivational	Noun Form
agreeable	-	agree	able	Derivational	Adjective Form

OUTPUT :

All words are normalized to agree for consistent sentimental analysis.

Q3) MORPHOLOGY--BASED NORMALIZATION:-

A news analytical system process govern - government and governance. The system should map all word forms to the same lexical representation.

INPUT WORDS:-

- Govern
- Government
- Governance

MORPHOLOGICAL ANALYSIS:-

WORD	ROOT	APPR	Type	Normalized
govern	govern	-	Base	govern
government	govern	ment	Derivational	govern
governance	govern	ance	Derivational	govern

OUTPUT:-

All words are normalized to the govern for document clustering.

4. MORPHOLOGICAL PASSING:-

A document classification system process activate, activation, and reactivation. It must identify prefix, suffix, and the root word to improve semantic grouping.

INPUT WORDS:-

- Active
- Activation
- Reactivation

MORPHOLOGICAL ANALYSIS:-

WORD	PREFIX	ROOT	SUFFIX	TYPE
active	-	activate	-	Base Form
activation	-	activate	ion	Derivational
reactivation	re	activate	ion	Derivational

OUTPUT:-

All words are normalized to active

5. INFLECTIONAL MORPHOLOGY:-

A search optimization module process create, creators, and creating. The system removes grammatical variations and stores the common base words.

INPUT WORDS:-

- Create
- Creators
- Creating

MORPHOLOGICAL ANALYSIS:-

WORD	ROOT	SUFFIX	GRAMMER	TYPE	NORMALIZED
Create	Create	-	Base form	Base	Create
Creators	Create	s	Third Person singular	Inflectional	create
Creating	Create	ing	Present participle	Inflectional	create

OUTPUT:-

All words are normalized to create for efficient search and informational retrieval.